Lecture 26: Recurrent neural networks part 2

Liberty Hamilton
April 30, 2026

Announcements

- For Stat248 - Final Project Instructions are on bCourses
- For Stat153 - Final Exam will be May 14 - 7-10pm, in this classroom
- No class next week, but we will have a final review session on Tuesday
- I will have regular office hours (Tuesday after class)
- Homework 5 is now optional/extra credit (3%)
- We will also have a little on pytorch with RNNs + exam review in Lab this week

Recap

- Recurrent neural networks (RNNs)
  - Appropriate for problems where data have sequential dependence
  - Trainable using backpropagation through time (BPTT)
    - This is like “unrolling” the recurrent network into a (very) deep feedforward network, and then applying backpropagation
  - RNNs are great for problems like language, which requires more sequential processing than vision

Today

- Gated RNNs
  - LSTMs
  - GRUs

More reading for today…

- The Unreasonable Effectiveness of Recurrent Neural Networks (Andrej Karpathy)
- Understanding LSTM Networks (Chris Olah)

The trouble with RNNs

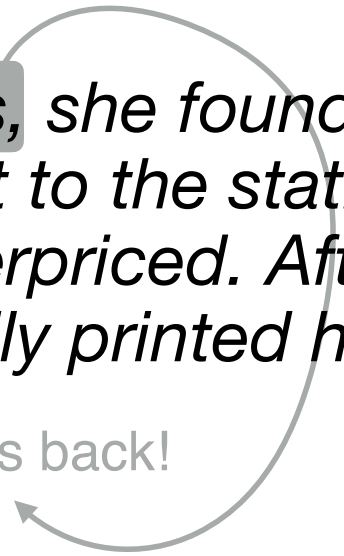
- Suppose we want to predict the last word:

  When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____

The trouble with RNNs

- Suppose we want to predict the last word:

  When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____



The trouble with RNNs

- RNNs suffer from the problems of **vanishing** and **exploding gradients**
- This problem is worse when backpropagating over a **long** sequence
  - Limits temporal dependence of functions that simple RNNs can learn

The trouble with RNNs

- Simple 1-D example:

$$h_{t+1} = \sigma(wh_t) \qquad \frac{\partial h_{t+1}}{\partial h_t} = w\sigma'(wh_t)$$

nonlinearity (e.g. tanh, sigmoid, maaybe ReLu)

The trouble with RNNs

- Simple 1-D example:

$$h_{t+1} = \sigma(wh_t) \qquad \frac{\partial h_{t+1}}{\partial h_t} = w\sigma'(wh_t)$$

$$\frac{\partial h_{t+2}}{\partial h_t} = w\sigma'(wh_{t+1})w\sigma'(wh_t)$$

The trouble with RNNs

- Simple 1-D example:

$$h_{t+1} = \sigma(wh_t)$$

$$\frac{\partial h_{t+1}}{\partial h_t} = w\sigma'(wh_t)$$

$$\frac{\partial h_{t+2}}{\partial h_t} = w\sigma'(wh_{t+1})w\sigma'(wh_t)$$

$$\frac{\partial h_{t+n}}{\partial h_t} = w^n \prod_{j=0}^{n-1} \sigma'(wh_{t+j})$$

The trouble with RNNs

- Simple 1-D example:

$$h_{t+1} = \sigma(wh_t)$$

$$\frac{\partial h_{t+1}}{\partial h_t} = w\sigma'(wh_t)$$

$$\frac{\partial h_{t+2}}{\partial h_t} = w\sigma'(wh_{t+1})w\sigma'(wh_t)$$

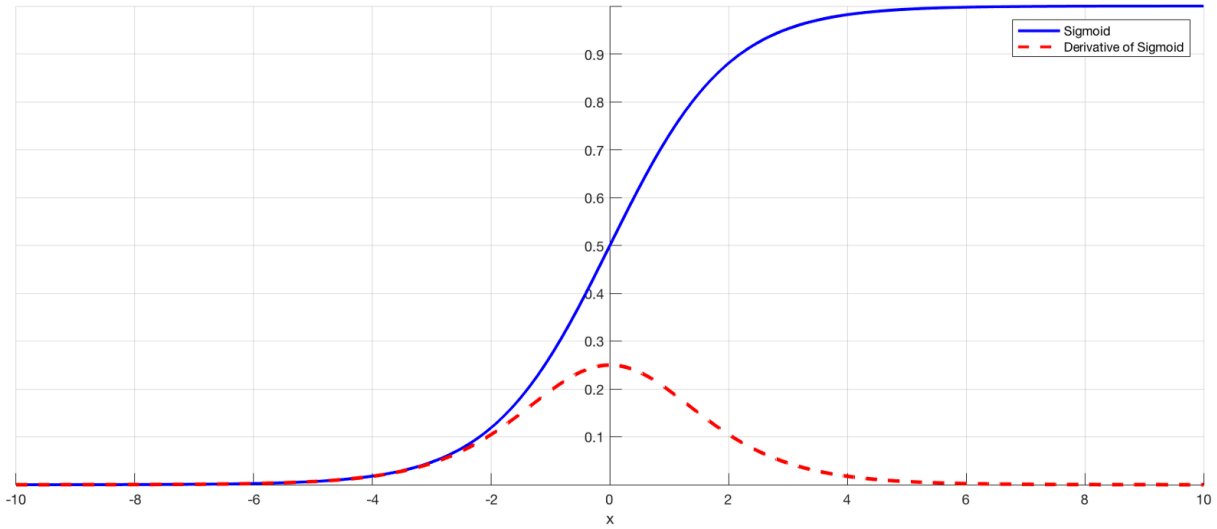
$$\frac{\partial h_{t+n}}{\partial h_t} = w^n \prod_{j=0}^{n-1} \sigma'(wh_{t+j})$$

really bad

also bad

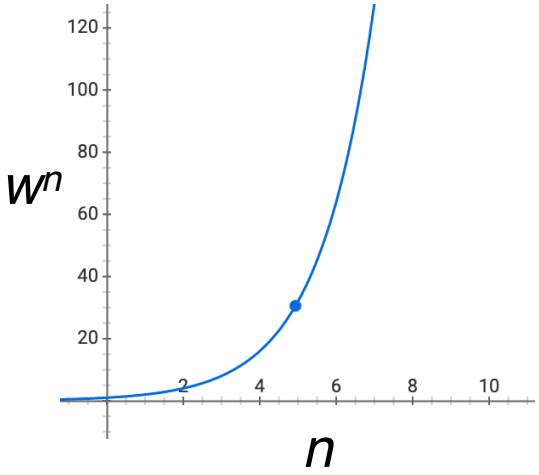
The trouble with RNNs

- **Vanishing gradients** occur if
  - n large, $|w| < 1$, or activations h_t far from 0



The trouble with RNNs

- **Exploding gradients** occur if
  - n large, |w| > 1



The trouble with RNNs

- How can these things be solved?
  - By breaking the chains of (1) weight multiplications, and (2) nonlinearities

$$\frac{\partial h_{t+n}}{\partial h_t} = w^n \prod_{j=0}^{n-1} \sigma'(w h_{t+j})$$

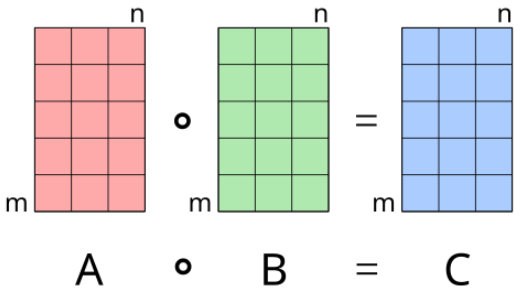
- Enter: ***Gated Recurrent Networks***

But first, notation

- Matrix Multiplication (*)
- “Hadamard product” (∘)
  - Element-wise multiplication
  - Notice the participating matrices & output all have the same shape

$$\begin{bmatrix} 0.8 & 0.1 \\ -0.9 & 0.2 \end{bmatrix} * \begin{bmatrix} 0.3 \\ -0.6 \end{bmatrix} = \begin{bmatrix} 0.18 \\ -0.39 \end{bmatrix}$$

$$\begin{bmatrix} 0.8 \\ -0.9 \end{bmatrix} \circ \begin{bmatrix} 0.3 \\ -0.6 \end{bmatrix} = \begin{bmatrix} 0.24 \\ -0.54 \end{bmatrix}$$

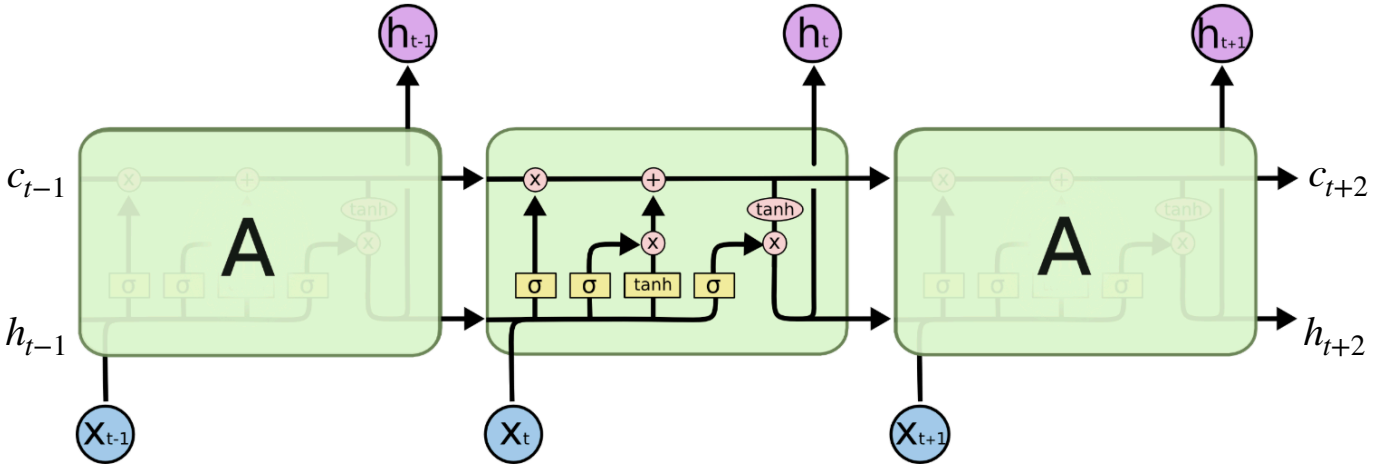


LSTM

- **L**ong **S**hort-**T**erm **M**emory network
- The LSTM was introduced by Hochreiter & Schmidhuber in 1997
- One example of a **gated recurrent network**

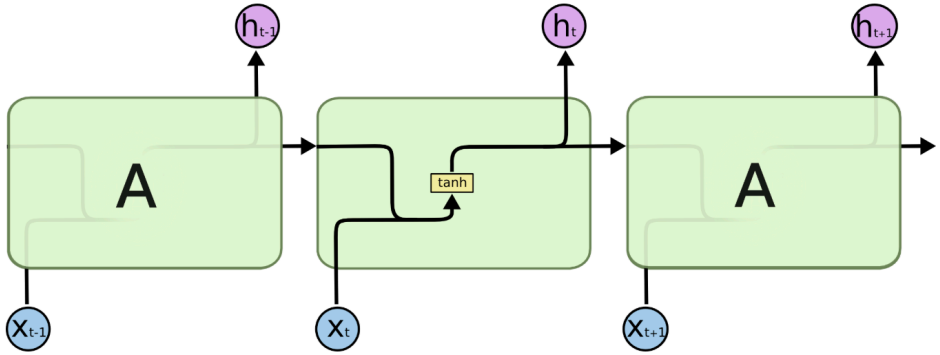
LSTM

- Basic idea:
- In addition to the **hidden state**, maintain a separate **cell state** (c_t) that does not pass through a non-linearity at each timestep

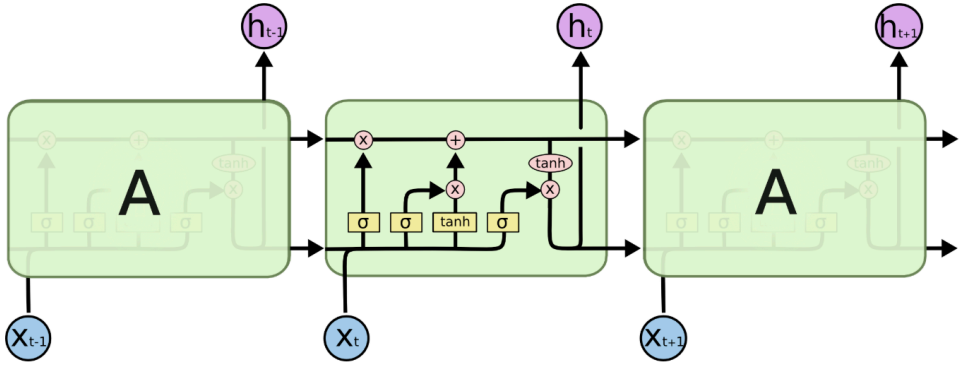


The repeating module in an LSTM contains four interacting layers.

LSTM vs Vanilla RNN



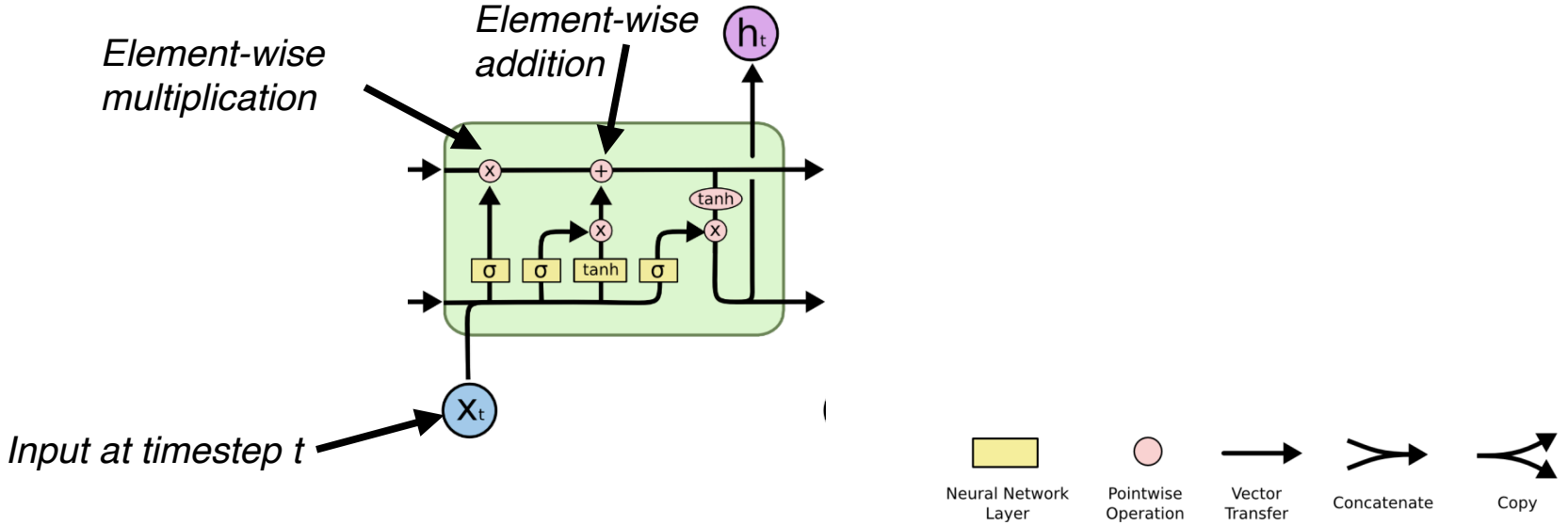
The repeating module in a standard RNN contains a single layer.



The repeating module in an LSTM contains four interacting layers.

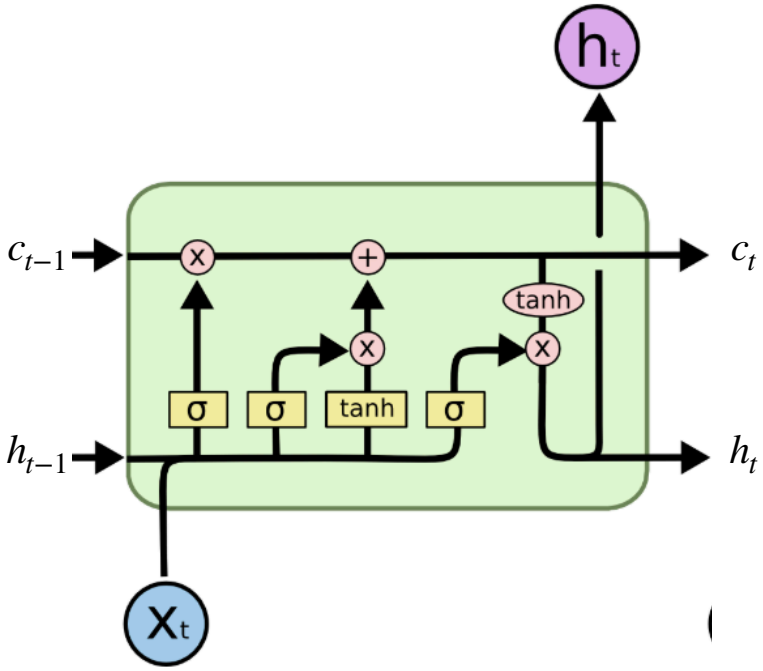
LSTM

- Basic idea:
- In addition to the **hidden state**, maintain a separate **cell state** (c_t) that does not pass through a non-linearity at each timestep



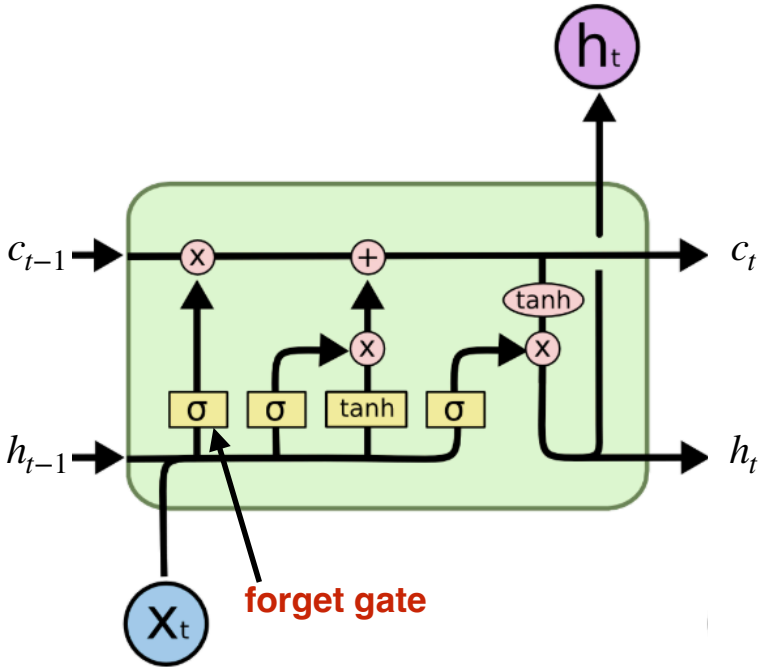
LSTM

- The LSTM uses a set of **gates** that can open or close to allow information to flow
  - Each gate is computed by *learning* a function on x_t and h_{t-1}, and applying sigmoid



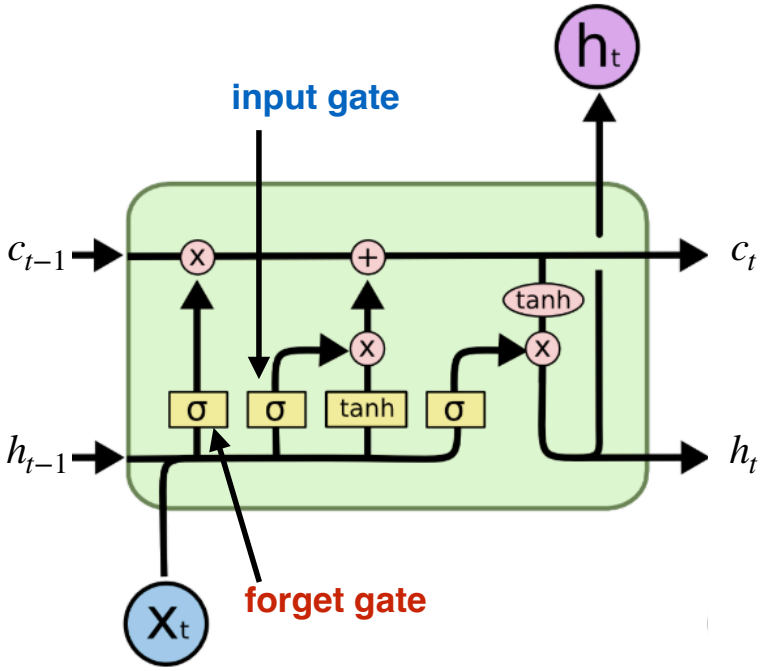
LSTM

- The LSTM uses a set of **gates** that can open or close to allow information to flow
  - Each gate is computed by *learning* a function on x_t and h_{t-1}, and applying sigmoid
  - The **forget gate** controls how information is removed from the cell state



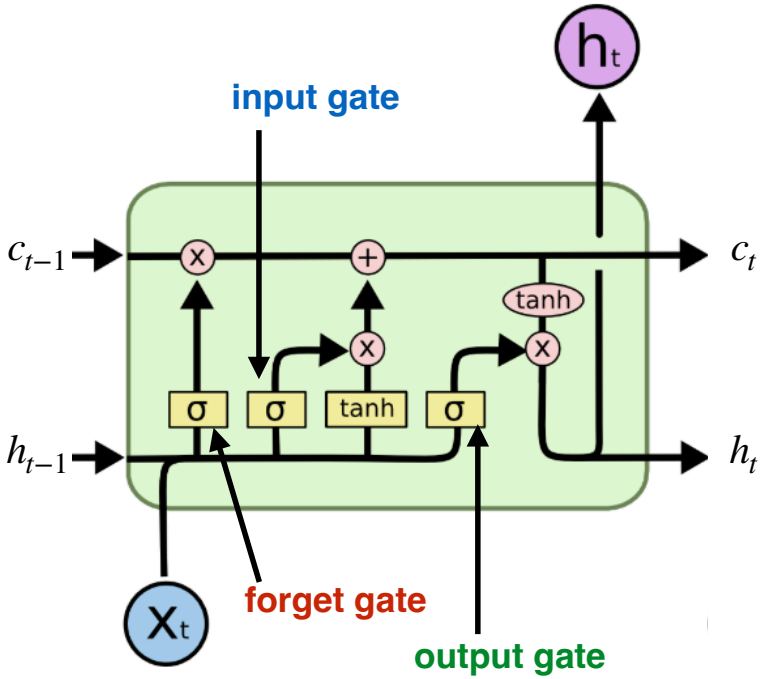
LSTM

- The LSTM uses a set of **gates** that can open or close to allow information to flow
  - Each gate is computed by *learning* a function on x_t and h_{t-1}, and applying sigmoid
  - The **forget gate** controls how information is removed from the cell state
  - The **input gate** controls how information is added to the cell state



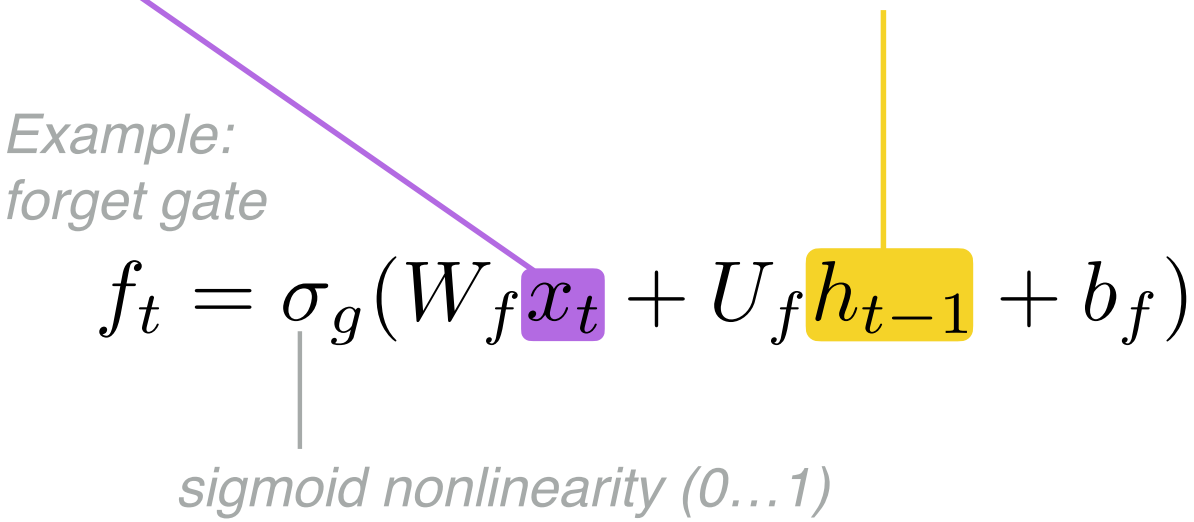
LSTM

- The LSTM uses a set of **gates** that can open or close to allow information to flow
  - Each gate is computed by *learning* a function on x_t and h_{t-1}, and applying sigmoid
  - The **forget gate** controls how information is removed from the cell state
  - The **input gate** controls how information is added to the cell state
  - The **output gate** controls how information from the cell state becomes output



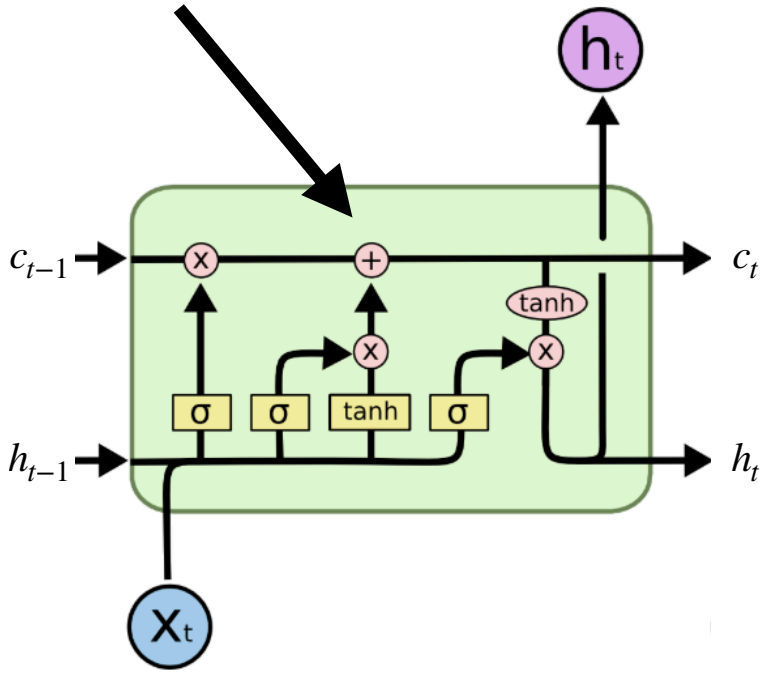
LSTM

- Each gate is computed as a learnable function of: **current input** and **previous hidden state**



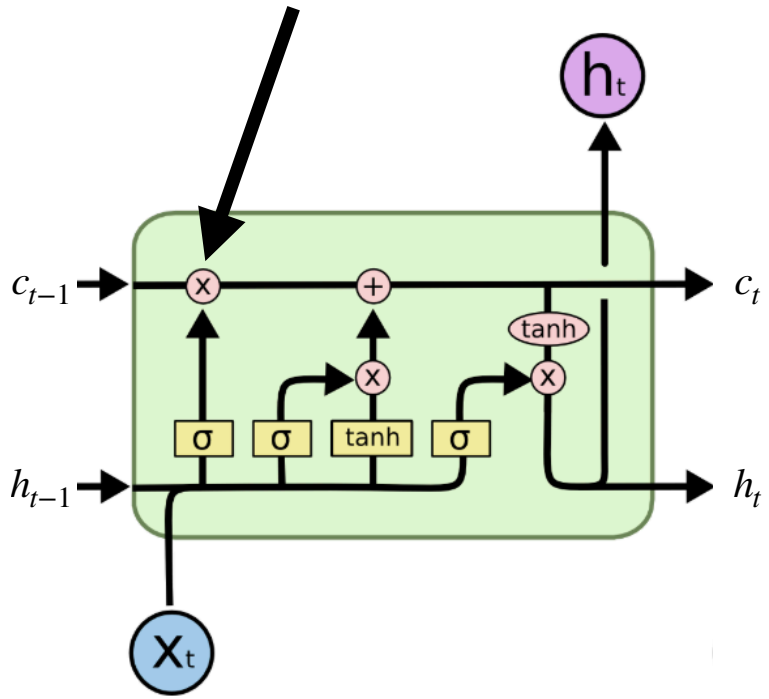
LSTM

- Information can be **added to** the cell state at each timestep



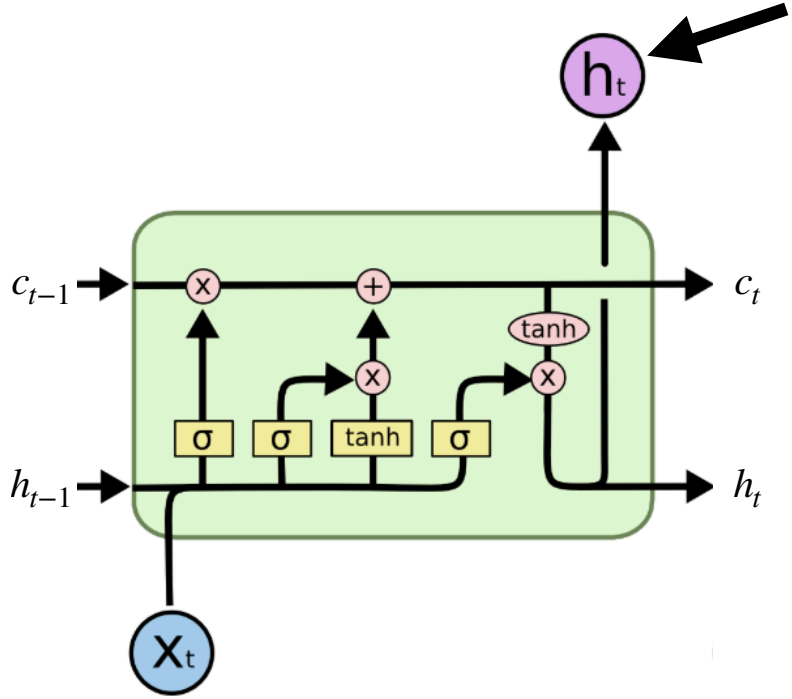
LSTM

- Information can also be **removed from** the cell state (or “forgotten”) at each timestep



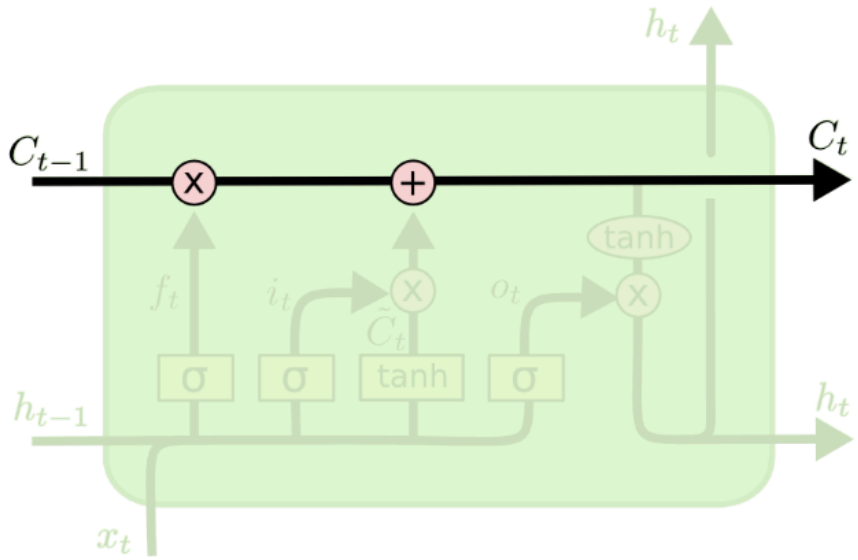
LSTM

- The new cell state is then used to construct the **output** “hidden” state



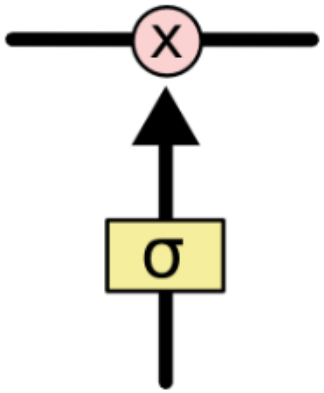
LSTM - Cell state c_t

- Like a conveyor belt - easy for info to flow unchanged



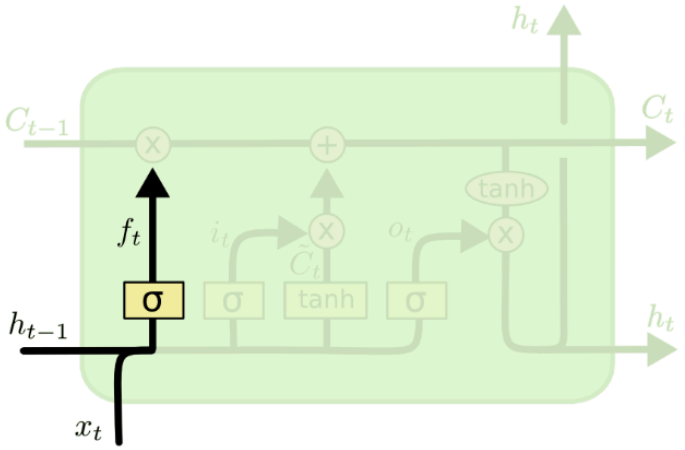
LSTM - gates

- Gates are a way to let information through
- **Sigmoid neural net layer** and a **pointwise multiplication**
- Sigmoid outputs [0,1]
  - let **nothing** (0) or
  - **everything** (1) through



LSTM - forget gate

- The **forget gate** is multiplied by the **previous cell state** and then added to the **proposed cell state** times the **input gate**

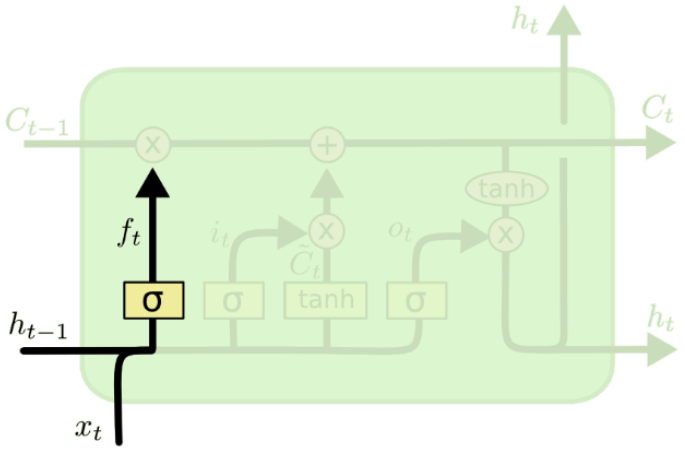


forget gate: $f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f)$

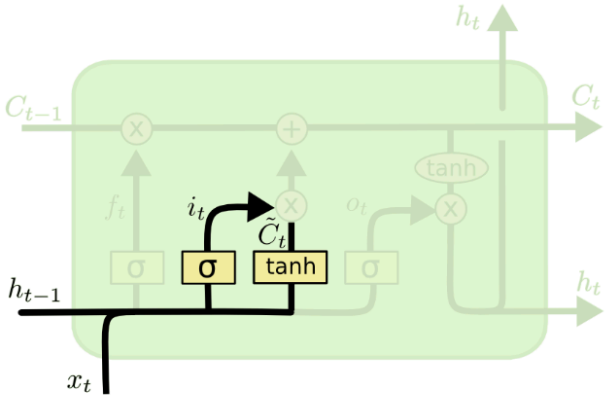
new cell state: $c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$

LSTM - forget gate

- Why do we need this?
- "***The woman*** walked into the bookstore cafe, enjoying the pleasant smells of coffee, cinnamon pastries, and old books. ***She*** ordered a latte. After finishing, she paid and left. **A man** sat down at the same table. ***He*** opened a book."



LSTM - input gate



- The **input gate** is multiplied by a **proposed cell state** and then added to the **previous cell state**

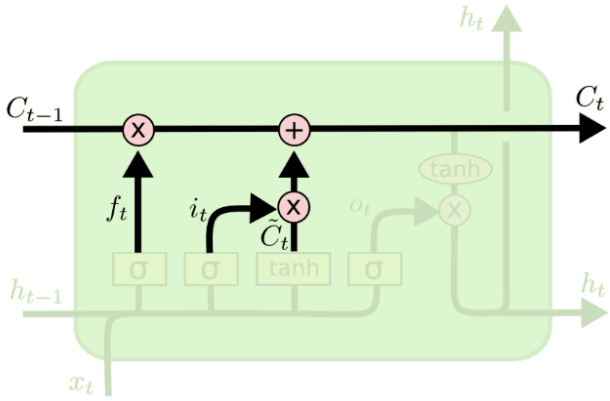
input gate: $$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i)$$

proposed cell state: $$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

new cell state: $$c_t = \cdots + i_t \circ \tilde{c}_t$$

element-wise or "Hadamard" product

LSTM - input gate



- The **input gate** is multiplied by a **proposed cell state** and then added to the **previous cell state**

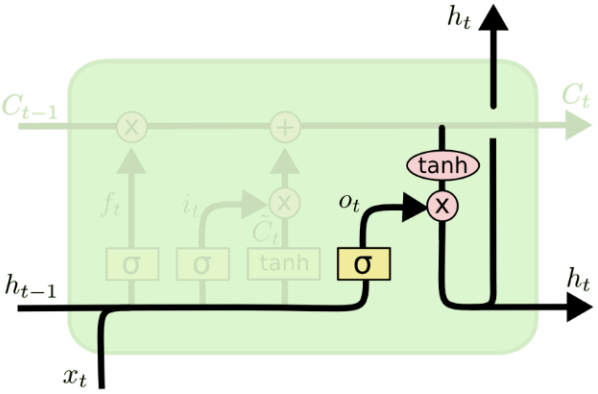
input gate: $$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i)$$

proposed cell state: $$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

new cell state: $$c_t = \cdots + i_t \circ \tilde{c}_t$$

element-wise or "Hadamard" product

LSTM - output gate



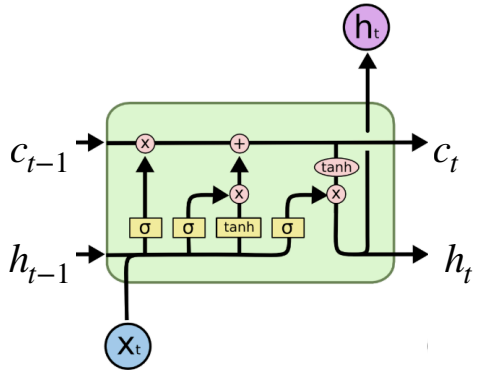
- The **output gate** is multiplied by the **current cell state** (with non-linearity applied) to form the **new hidden state**

output gate: $o_t = \sigma_g(W_o x_t + U_o h_{t-1} + b_o)$

new hidden state: $h_t = o_t \circ \tanh(c_t)$

- Might output information relevant to what's coming next (is subject singular or plural to help verb conjugation)

LSTM



- All together now!
  - Input gate:
  - Forget gate:
  - Output gate:
  - Proposed cell:
  - New cell state:
  - New hidden state:

$$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i)$$

$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f)$$

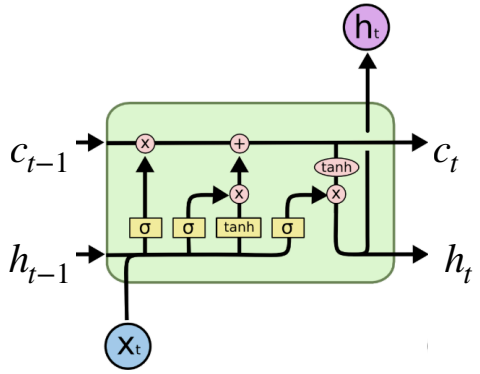
$$o_t = \sigma_g(W_o x_t + U_o h_{t-1} + b_o)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$$

$$h_t = o_t \circ \tanh(c_t)$$

LSTM



- All together now!
  - Input gate:
  - Forget gate:
  - Output gate:
  - Proposed cell:
  - New cell state:
  - New hidden state:

$$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i)$$

$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f)$$

$$o_t = \sigma_g(W_o x_t + U_o h_{t-1} + b_o)$$

fitted values

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$$

$$h_t = o_t \circ \tanh(c_t)$$

LSTM

- How does the LSTM solve the problems of exploding & vanishing gradients?
- Similar to our earlier 1-D example:

$$c_{t+1} = f_{t+1} \circ c_t + \dots \qquad \frac{\partial c_{t+1}}{\partial c_t} \approx f_{t+1}$$

LSTM

- How does the LSTM solve the problems of exploding & vanishing gradients?
- Similar to our earlier 1-D example:

$$c_{t+1} = f_{t+1} \circ c_t + \dots$$

$$\frac{\partial c_{t+1}}{\partial c_t} \approx f_{t+1}$$

$$\frac{\partial c_{t+n}}{\partial c_t} \approx \prod_{j=1}^{n} f_{t+j}$$

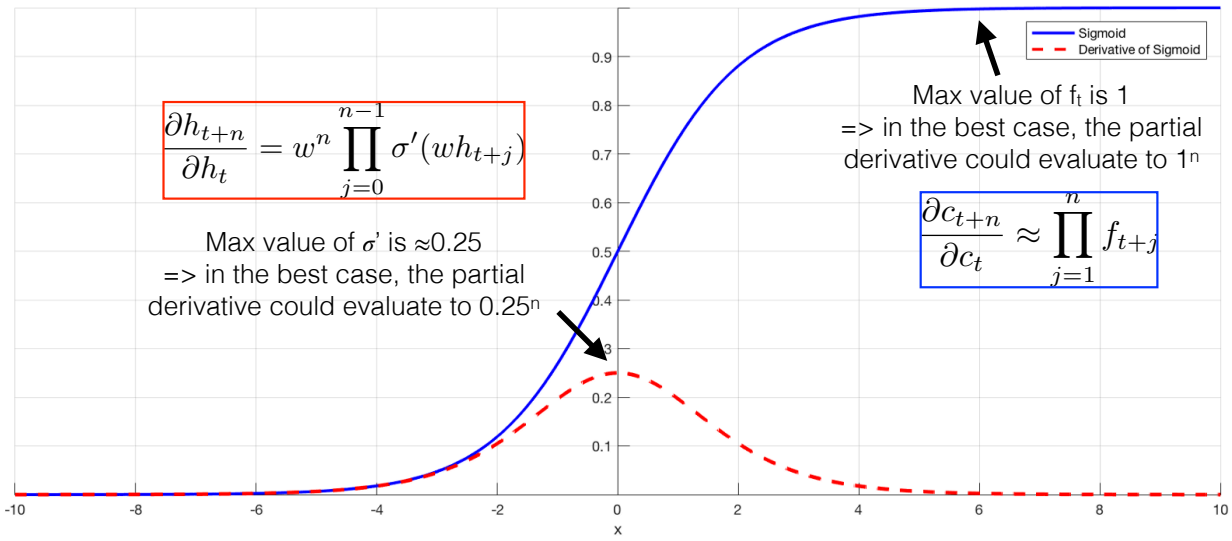
LSTM

- How does the LSTM solve the problems of exploding & vanishing gradients?
- Similar to our earlier 1-D example:

$$c_{t+1} = f_{t+1} \circ c_t + \dots \qquad \frac{\partial c_{t+1}}{\partial c_t} \approx f_{t+1}$$

can shrink (a bit) but not explode! $\longrightarrow$ $\frac{\partial c_{t+n}}{\partial c_t} \approx \prod_{j=1}^{n} f_{t+j}$

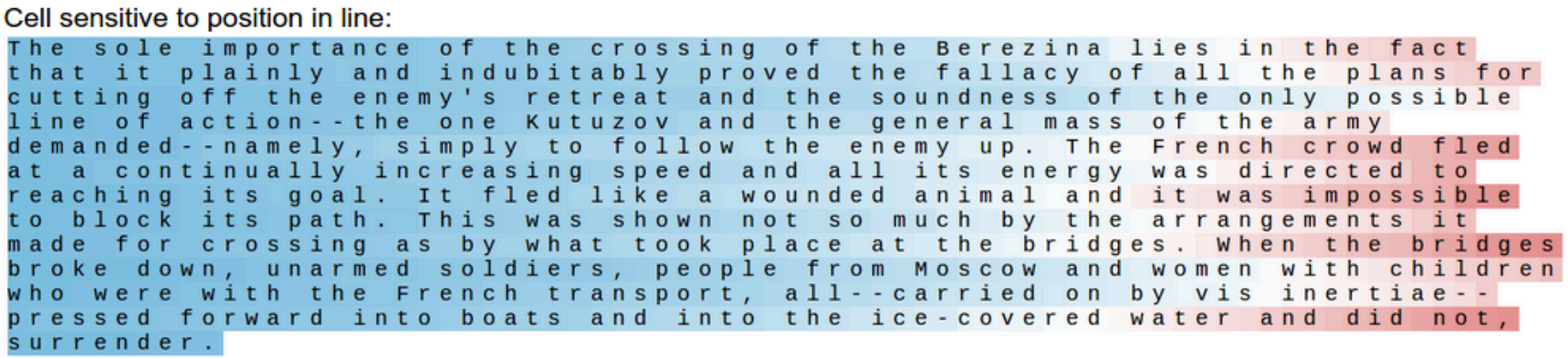
LSTM



- How does the LSTM solve the problems of exploding & vanishing gradients?
- Similar to our earlier 1-D example:

LSTM visualization

- Train an LSTM to predict the **next character** in a sequence from the previous characters
  - Network trained either on text (e.g. *War and Peace*) or C++ code (e.g. the Linux kernel)
- Visualizing the **cell state values** (for a few units) at each character can show a bit of what the network has learned!



From: http://karpathy.github.io/2015/05/21/rnn-effectiveness/

LSTM visualization

- Train an LSTM to predict the **next character** in a sequence from the previous characters
  - Network trained either on text (e.g. *War and Peace*) or C++ code (e.g. the Linux kernel)
- Visualizing the **cell state values** (for a few units) at each character can show a bit of what the network has learned!

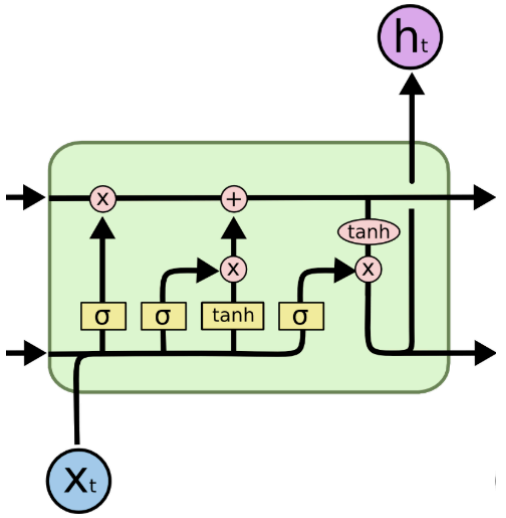
Cell that robustly activates inside if statements:

```
static int __dequeue_signal(struct sigpending *pending, sigset_t *mask,
        siginfo_t *info)
{
  int sig = next_signal(pending, mask);
  if (sig) {
    if (current->notifier) {
      if (sigismember(current->notifier_mask, sig)) {
        if (!(current->notifier)(current->notifier_data)) {
          clear_thread_flag(TIF_SIGPENDING);
          return 0;
        }
      }
    }
    collect_signal(sig, pending, info);
  }
  return sig;
}
```

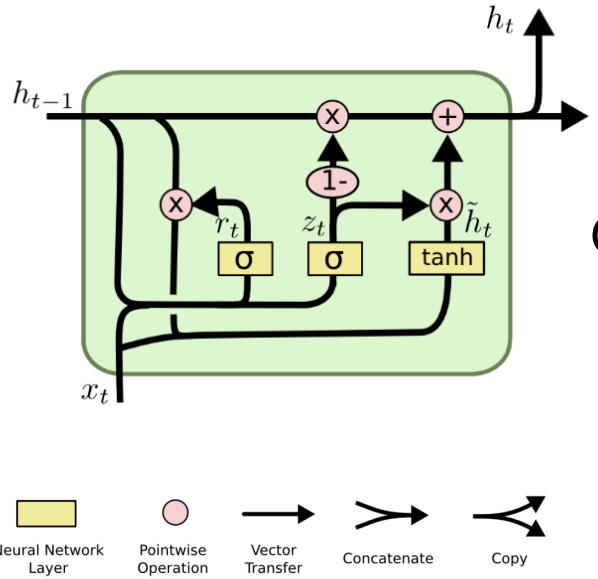
From: http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Gated Recurrent Unit

- Kyunghyun Cho et al., 2014 and Junyoung Chung et al. 2014
- Combines forget and input gates into single "update gate"
- Merges cell state and hidden state
- Simpler than standard LSTMs, fewer parameters to fit!



LSTM



GRU

Applications of LSTMs/RNNs

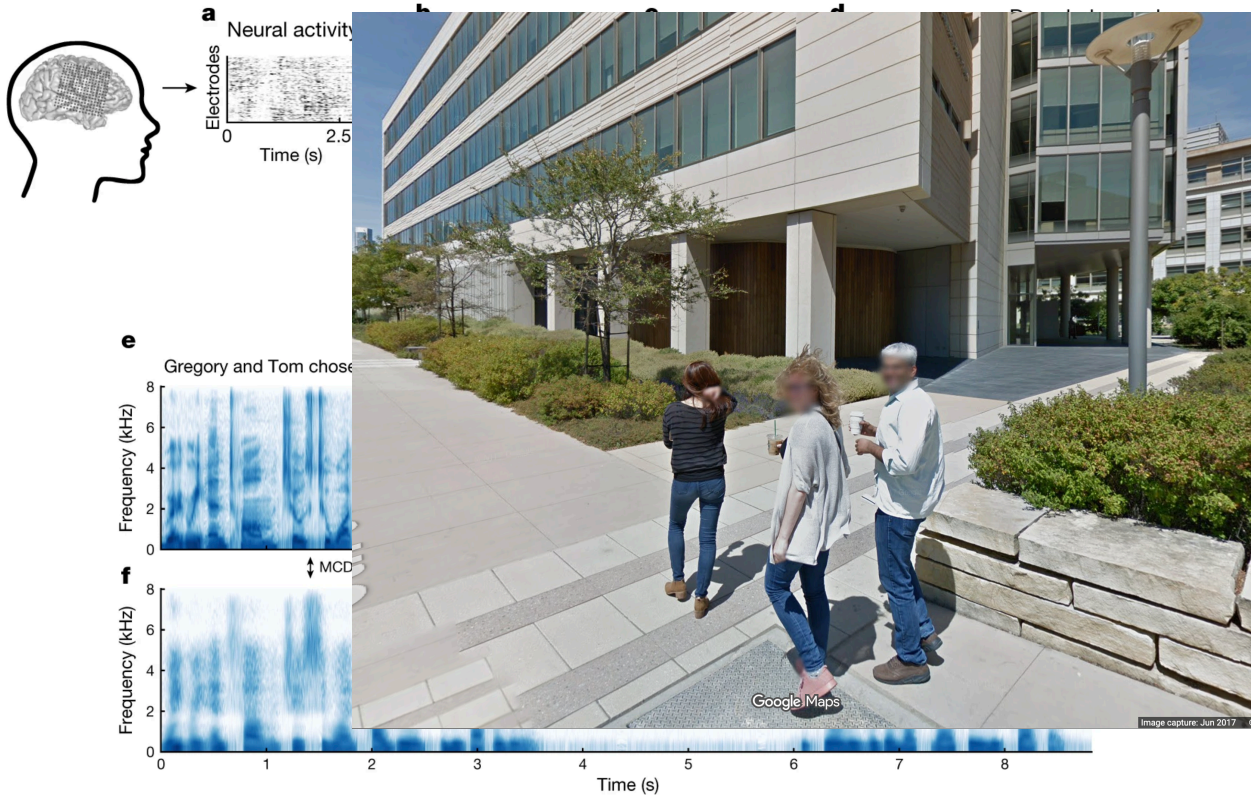
- Anumanchipalli et al. 2019
- Synthesizing speech from brain data
- One of the foundational papers for speech BCI
- Used bidirectional LSTM

Article | Published: 24 April 2019

Speech synthesis from neural decoding of spoken sentences

Gopala K. Anumanchipalli, Josh Chartier & Edward F. Chang

Nature **568**, 493–498 (2019) | Cite this article



Applications of LSTMs/RNNs

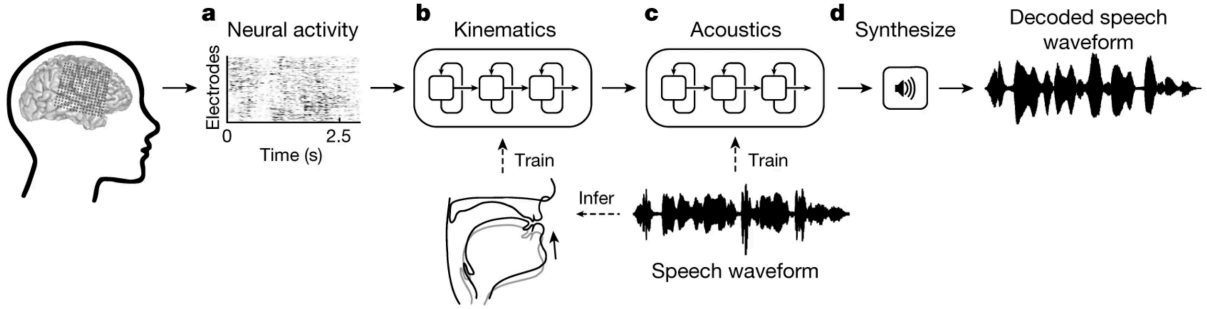
- Anumanchipalli et al. 2019
- Synthesizing speech from brain data
- One of the foundational papers for speech BCI
- Used bidirectional LSTM

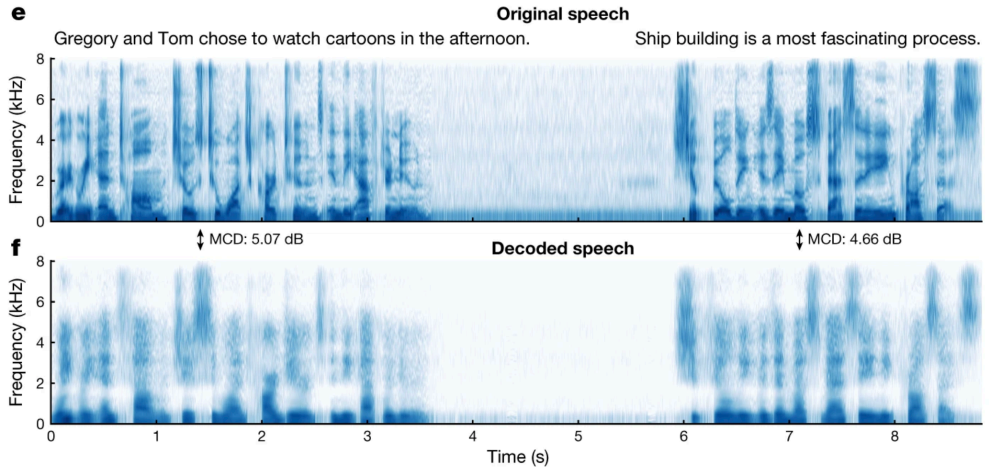
Article | Published: 24 April 2019

Speech synthesis from neural decoding of spoken sentences

Gopala K. Anumanchipalli, Josh Chartier & Edward F. Chang

Nature **568**, 493–498 (2019) | Cite this article






Speech synthesis from
neural decoding of spoken sentences
UCSF

Applications of LSTMs/RNNs

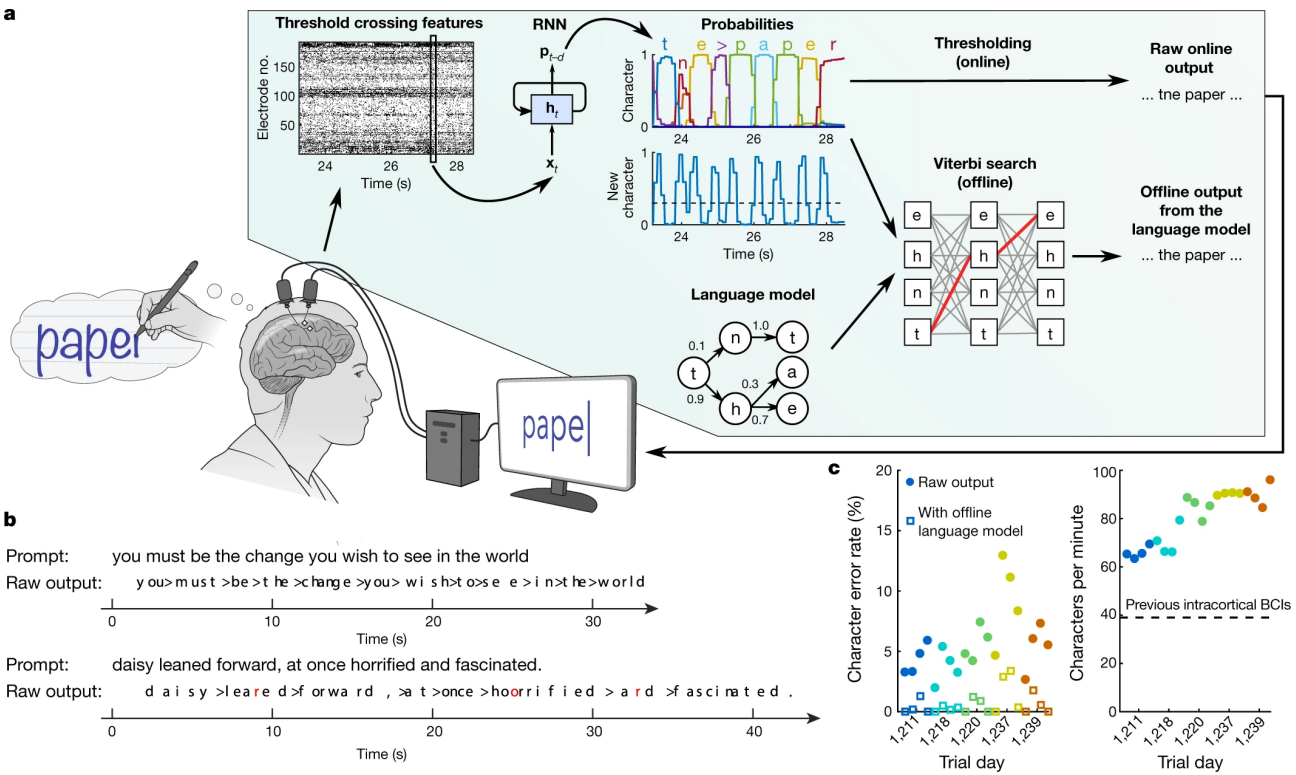
- Handwriting decoding (Willett et al. 2021)
  - Two layer, GRU to convert neural activity into time series of character probabilities
  - GRU outperformed simple HMM decoder

Article | Published: 12 May 2021

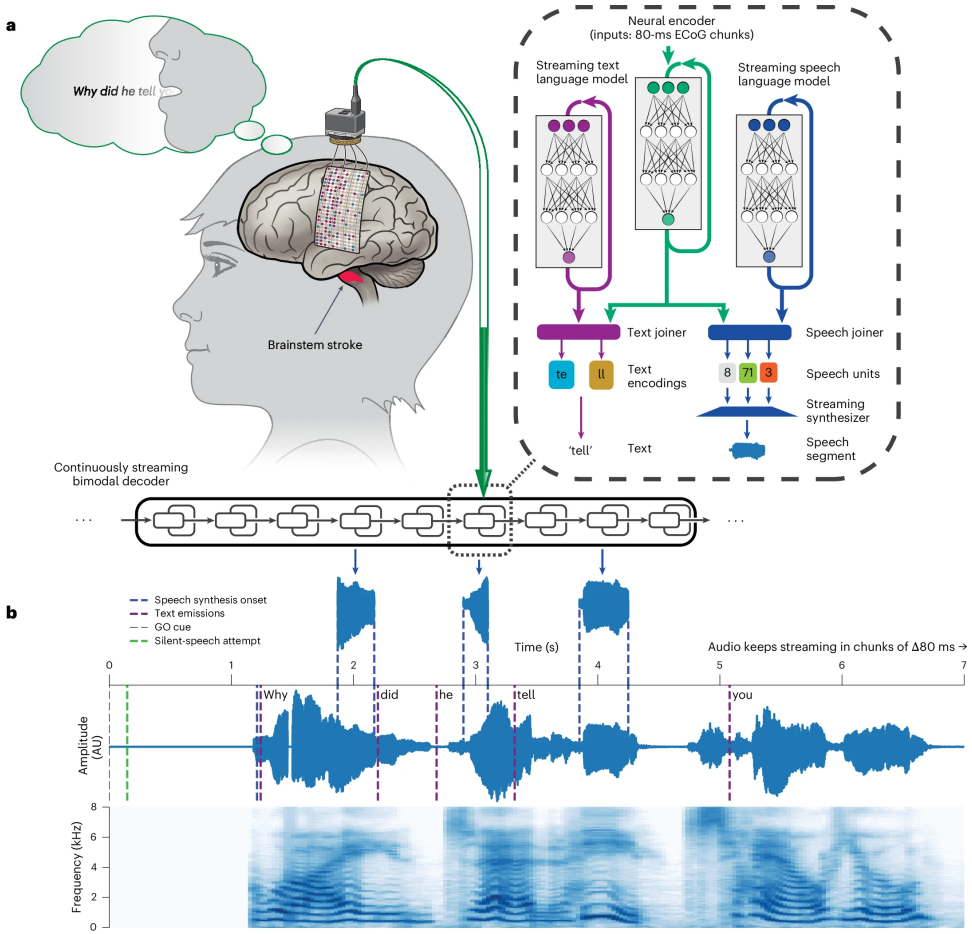
High-performance brain-to-text communication via handwriting

Francis R. Willett ✉, Donald T. Avansino, Leigh R. Hochberg, Jaimie M. Henderson & Krishna V. Shenoy

Nature **593**, 249–254 (2021) | Cite this article



Applications of LSTMs/RNNs



Technical Report | Published: 31 March 2025

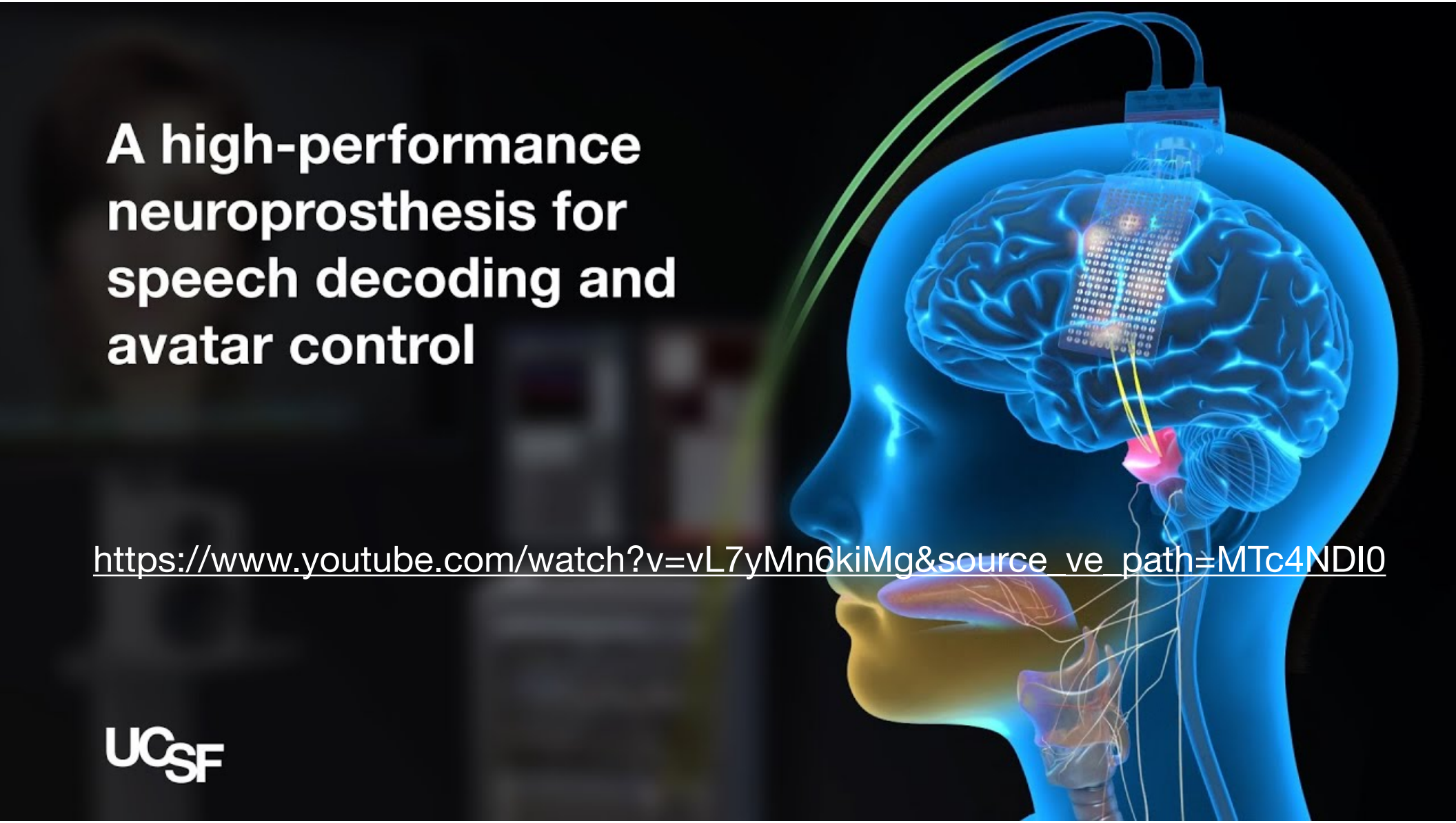
A streaming brain-to-voice neuroprosthesis to restore naturalistic communication

Kaylo T. Littlejohn, Cheol Jun Cho, Jessie R. Liu, Alexander B. Silva, Bohan Yu, Vanessa R. Anderson, Cady M. Kurtz-Miott, Samantha Brosler, Anshul P. Kashyap, Irina P. Hallinan, Adit Shah, Adelyn Tu-Chan, Karunesh Ganguly, David A. Moses, Edward F. Chang ✉ & Gopala K. Anumanchipalli ✉

Nature Neuroscience **28**, 902–912 (2025) | Cite this article

- Littlejohn et al. 2025
- Real-time decoding of speech from a patient with severe paralysis and anarthria
- Architecture was RNN-T (RNN-transducer)

A high-performance neuroprosthesis for speech decoding and avatar control



https://www.youtube.com/watch?v=vL7yMn6kiMg&source_ve_path=MTc4NDI0

UCSF

So what have we learned?

- Measures of dependence in time series signals (autocovariance/autocorrelation)
- Power spectral analysis and time-frequency analysis
- Linear and nonlinear methods for predicting time series data
  - Regression, ARIMA, state space analyses
- Cross-validation for time series and important considerations
- Modern methods for time series and sequence data (CNNs, RNNs, LSTMs)
- ***Time series are everywhere!***

That’s all!

Course evaluations

- Please take some time to complete your course evaluations!
- I’d love to hear what you liked, what could be improved